



Séance 5

Processus ERLANG

Communication entre les processus

Le projet

Exemples : jeu en ligne (jeu de rôle), plateforme sociale (rencontre, partage),

Modalités projet ERLANG 1/2

Groupe de 5 étudiants.

Heures fléchées dans l'emploi du temps

- 05/05 2 heures encadrées
- 11/05 2 heures autonomie
- 12/05 2 heures encadrées
- 18/05 2 heures autonomie
- 28/05 4 heures : recette en amphi le 28/05

Présentation 15 minutes avec ppt et démonstration.

Modalités projet ERLANG 2/2

Problématique technique (soyez rigoureux) :

- Utiliser les processus et la communication entre processus.
- Mettre en œuvre sur une (plusieurs machines en réseau) un réseau de BEAM qui communiquent entre elles.
- Modèle imposé : client(s) / serveur.

Problématique fonctionnelle (soyez originaux) :

- A choisir entre vous
- A soumettre pour validation

Exemples : jeu de rôle textuel en ligne, plateforme sociale,

Evaluation projet ERLANG 1/3

Problématique technique basique :

- Mise en œuvre des processus et de la communication entre les processus.
- Réalisation sur une (plusieurs machines en réseau) un réseau de BEAM qui communiquent entre elles.

Problématique technique orientée projet :

- Démontrer que le projet est bien en mode client(s) / serveur.
- A minima le serveur doit être sous ERLANG
- Vous pouvez utiliser le module `gen_tcp` pour mettre en oeuvre des clients dans un autre langage (et sur d'autres machines).
- Pour la BDD : utiliser MNESIA

Evaluation projet ERLANG 2/3

Sur la base d'un ppt (tout le monde présente) :

1. La charte projet : Description du besoin
2. Un schéma d'architecture statique / dynamique
Pensez à qualifier les requêtes et les réponses

ex : requête demande de connexion login/mdp"
réponse : autorisation de connexion

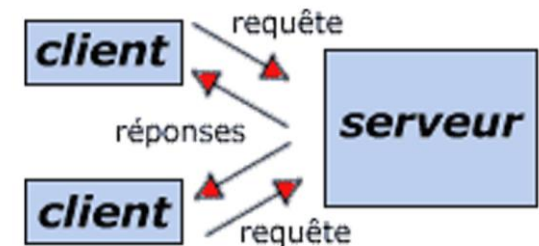


Schéma de fonctionnement d'un système client/serveur (commentcamarche.net - © 2003 Pillou - GNU FDL)

- Depuis le serveur : envoi d'une question à tous les clients
 - Client vers serveur : première réponse envoyée comptabilise les points
3. Deux ou trois scénarios de connexion
 4. Les composants logiciels utilisés (bibliothèques)
 5. La charte projet (reprise) : Qui (a) fait quoi

Evaluation projet ERLANG 3/3

La démo :

- Si elle n'est pas préparée il y a de fortes chances pour qu'il y ait un problème technique.
- Compte tenu des temps de passage contraints, une démo ratée ne sera pas reprise.

Nota : par précaution, vous pouvez "filmer" le déroulement de votre démo.

Le temps de passage est de 10 minutes par projet (environ 2 min / élèves).

L'ordre de passage vous sera communiqué le jour précédent.

En plus

ets / mnésia
gen_tcp

Stockage d'information

Utilisation du module ets, fonctions

Le module ets pour ERLANG Term Storage permet l'enregistrement et la relecture de termes à partir de clés.

ets:new/2 est utilisé pour créer une table,

- le premier argument est le nom de la table

ets:insert/2 est utilisé pour insérer dans une table créée

- le premier argument est le nom de la table,
- le second le tuple clé, valeur

ets:lookup/2 est utilisé pour récupérer une information à partir d'une clé

- le premier argument est le nom de la table,
- le second la clé

Stockage d'information

Utilisation du module ets, exemple

```
-module(memo).  
-export([start0/0, store_data/2, fetch_data/2]).  
start0() ->  
    Table = ets:new(ma_table, [named_table, public]),  
    store_data(Table, {cle1, "valeur"}),  
    io:format("Donnée récupérée : ~p~n", [fetch_data(Table, cle1)]),  
    io:format("Dans la table : ~p~n", [Table]).
```

```
store_data(Table, {Key, Value}) ->  
    ets:insert(Table, {Key, Value}).
```

```
fetch_data(Table, Key) ->  
    case ets:lookup(Table, Key) of  
        [{Key, Value}] -> Value;  
        [] -> undefined  
    end.
```

Module mnesia (à préférer)

Pour prendre en main la base de données mnesia, utiliser l'exemple du chapitre 2

https://www.erlang.org/doc/apps/mnesia/mnesia_chap2.html

Pour prendre en main la base de données mnesia, utiliser l'exemple du chapitre 2

Module gen_tcp

Pour communiquer entre 2 processus sur 2 machines distantes en réseau via des sockets TCP/IP.

Exemple de mise en oeuvre des fonctions :

- **listen** : serveur / création d'un socket d'écoute à partir d'une adresse et d'un
- **accept** : création d'un socket de service entre le serveur et le client
- **send** : envoi de données en mode passif
- **receive** : réception de données en mode passif
- **close** : fermeture de la connexion

Exemple d'utilisation : projet 1/4

2 modules avec des rôles différents

- serveur.erl
- client.erl

Serveur --- PRINCIPE --- Client
Machine sur le réseau + port

(1) Écoute active

Connexion au serveur (2)

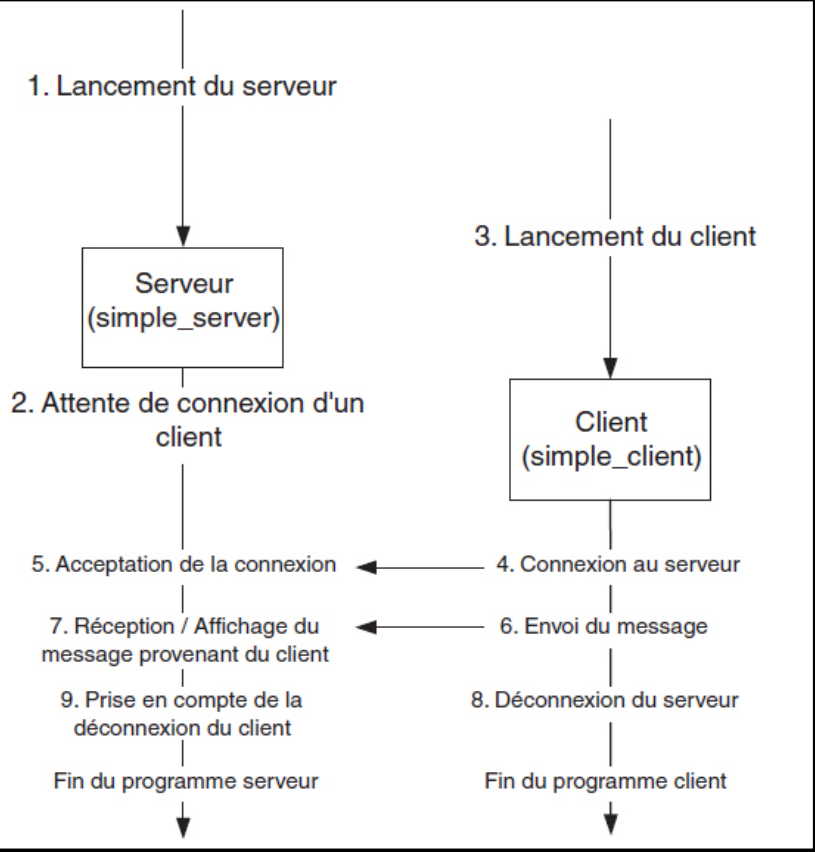
Création d'un socket de service

(3) acceptation

Échange sur le socket de service

Envoi message (4)

(5) Réception du message



```
1 -module(serveur).
2 -export([serveur/0]).
3
4 serveur() ->
5     {ok,LSock} = gen_tcp:listen(5678, [binary, {packet,0}, {active, false}]),
6     {ok, Sock} = gen_tcp:accept(LSock),
7     {ok, Bin} = do_recv(Sock, []),
8     ok = gen_tcp:close(Sock),
9     ok = gen_tcp:close(LSock),
10    Bin.
11
12 do_recv(Sock, Bs) ->
13     case gen_tcp:recv(Sock,0) of
14         {ok, B} -> do_recv(Sock,[Bs, B]);
15         {error, closed} -> {ok, list_to_binary(Bs)}
16     end.
```

Module : client.erl

```
1 -module(client).
2 -export([client/0]).
3
4 client() ->
5     TheHost = "localhost",
6     {ok, Sock} = gen_tcp:connect(TheHost, 5678, [binary, {packet, 0}]),
7     ok = gen_tcp:send(Sock, "Les données"),
8     ok = gen_tcp:close(Sock).
9
```

Module : serveur.erl

```
1 -module(serveur).
2 -export([serveur/0]).
3
4 serveur() ->
5     {ok,LSock} = gen_tcp:listen(5678, [binary, {packet,0}, {active, false}]),
6     {ok, Sock} = gen_tcp:accept(LSock),
7     {ok, Bin} = do_recv(Sock, []),
8     ok = gen_tcp:close(Sock),
9     ok = gen_tcp:close(LSock),
10    Bin.
11
12 do_recv(Sock, Bs) ->
13     case gen_tcp:recv(Sock,0) of
14         {ok, B} -> do_recv(Sock,[Bs, B]);
15         {error, closed} -> {ok, list_to_binary(Bs)}
16     end.
```

Fonction listen - paramètres

Lsock : crée un socket d'écoute sur le port 5678

3 options dans une liste :

binary : réception binaire

{packet, 0} : données reçues sans en-tête de paquet

{active, false} : mode passif, appel explicite de recv/2

Retourne ok et crée le socket ou error avec la raison

Fonction accept - paramètre

Lsock le socket d'écoute

Retourne ok et crée Sock le descripteur du socket pour la connexion acceptée

Fonction recv – paramètres

Sock : réception en mode passif sur ce socket

0 : pas de limitation au nombre de d'octets

Module : client.erl

```
1 -module(client).
2 -export([client/0]).
3
4 client() ->
5     TheHost = "localhost",
6     {ok, Sock} = gen_tcp:connect(TheHost, 5678, [binary, {packet, 0}]),
7     ok = gen_tcp:send(Sock, "Les données"),
8     ok = gen_tcp:close(Sock).
9
```

Fonction connect - paramètres

TheHost : le nom ou l'adresse IP du poste sur le réseau, ici le même poste

5678 : le port de connexion

3 options dans une liste :

binary : réception binaire

{packet, 0} : données reçues sans en-tête de paquet

Retourne ok et crée le socket de service

Sock pour l'envoi

ou error avec la raison

Fonction send - paramètres

Sock : le socket utilisé pour la connexion de service

"Les données" : les données envoyées au serveur